# Arquitectura de Sistemas de Interfaz en React: Convergencia de Estado, Datos y Formularios

## Resumen Ejecutivo y Alcance

En la ingeniería de software moderna, específicamente dentro del ecosistema de React, la gestión del estado ha evolucionado desde un enfoque monolítico y centralizado hacia una arquitectura distribuida y especializada. La responsabilidad de gestionar los datos ya no recae únicamente en un almacén global en memoria (como Redux en sus primeras iteraciones), sino que se fragmenta estratégicamente entre el servidor, la URL, y gestores de formularios efímeros. Este informe técnico, dirigido a líderes técnicos y arquitectos de frontend, proporciona un análisis exhaustivo sobre la integración de tres pilares fundamentales para aplicaciones de datos intensivos: **React Hook Form (con validación Zod)**, **TanStack Table v8**, y la **Gestión de Estado en la URL (via nuqs)**.

El objetivo es trascender la implementación básica y explorar patrones de diseño avanzados, seguridad de tipos (type-safety) rigurosa con TypeScript, y estrategias de rendimiento para aplicaciones empresariales escalables. A través de este documento, desglosaremos cómo estas tecnologías no solo coexisten, sino que forman una "trinidad arquitectónica" que garantiza la integridad de los datos, la experiencia de usuario determinista y la mantenibilidad del código a largo plazo.

---

## 1. El Paradigma de la Gestión de Estado Distribuida

### 1.1 De la Memoria Volátil a la Persistencia Determinista

Históricamente, las Single Page Applications (SPAs) trataban al navegador como una caja negra donde el estado vivía y moría en la memoria JavaScript (useState, Context). Si bien esto ofrecía una interactividad fluida, introducía una fragilidad crítica: la incapacidad de compartir, marcar o restaurar el estado exacto de una aplicación simplemente copiando la URL.

La arquitectura moderna aboga por mover la "fuente de la verdad" desde la memoria del cliente hacia ubicaciones más resilientes y semánticas. Identificamos tres capas de estado que deben ser orquestadas:

| Tipo de Estado | Ubicación Principal | Herramienta Recomendada | Características |
|---|---|---|---|
| **Estado del Servidor** | Base de Datos / Caché HTTP | TanStack Query / SWR / Server Components | Asíncrono, compartido, requiere invalidación y re-validación. |
| **Estado de la URL** | Barra de Direcciones / Historial | nuqs / React Router | Persistente entre recargas, compartible, serializable. |
| **Estado del Formulario** | DOM / Referencias Locales | React Hook Form | Efímero, sucio (dirty), requiere validación síncrona/asíncrona. |
| **Estado de UI** | Memoria Componente | useState / useReducer | Interacciones visuales transitorias (modales, tooltips). |

El enfoque de este informe se centra en cómo eliminar la redundancia entre estas capas. Por ejemplo, el estado de paginación de una tabla no debería duplicarse en useState si ya existe conceptualmente en la URL. Del mismo modo, la validación de un formulario no debería reescribirse en el servidor si podemos compartir un esquema de **Zod**.

### 1.2 La Importancia de la Sincronización Bidireccional

La fricción en el desarrollo moderno a menudo surge en los límites (boundaries) entre estas capas. ¿Cómo aseguramos que la URL ?page=2 actualice la tabla, y que el clic en "Siguiente" en la tabla actualice la URL, sin causar bucles de renderizado o inconsistencias? La respuesta yace en el uso de primitivas que traten a la URL como un almacén de estado reactivo, un concepto que exploraremos a profundidad con la librería **nuqs** y los patrones de integración con **TanStack Table**.

---

## 2. Ingeniería de Formularios de Alto Rendimiento: React Hook Form y Zod

La gestión de formularios es una de las tareas más complejas en el desarrollo frontend debido a la necesidad de coordinar la entrada del usuario, la validación, el manejo de errores y el envío de datos, todo mientras se mantiene un rendimiento óptimo de renderizado.

### 2.1 Arquitectura de Componentes No Controlados

**React Hook Form (RHF)** se distingue de sus predecesores (como Formik) por su arquitectura basada en **componentes no controlados** y el uso de Proxies. En lugar de mantener el valor de cada input en el estado de React (lo que provocaría un re-renderizado del formulario completo con cada pulsación de tecla), RHF registra referencias (refs) a los elementos del DOM.[1]

Este enfoque tiene implicaciones profundas en el rendimiento:

1. **Aislamiento de Renderizado:** Escribir en un campo de texto no obliga a renderizar sus hermanos ni al contenedor padre, a menos que haya una dependencia explícita (como una validación cruzada que muestre un error en tiempo real).
2. **Suscripción Selectiva:** RHF permite que los componentes se suscriban a cambios específicos del estado del formulario (isDirty, isValid, errors) de manera granular, reduciendo la carga en el hilo principal.

### El Patrón de Registro y Tipado

La función register es el mecanismo mediante el cual vinculamos un elemento del DOM con la lógica de RHF. Sin embargo, en un entorno TypeScript estricto, el desafío es asegurar que los nombres de los campos (name) coincidan perfectamente con la estructura de datos esperada.

TypeScript

```
// Ejemplo conceptual de tipado estricto
const { register } = useForm<Inputs>();
// TypeScript lanzará un error si "firstName" no existe en Inputs
<input {...register("firstName")} />
```

## 2.2 Validación Declarativa con Zod

La validación manual es propensa a errores y difícil de mantener. **Zod** emerge como el estándar "de facto" para la definición de esquemas debido a su integración con TypeScript ("TypeScript-first"). Zod permite definir no solo la validación, sino la *forma* de los datos, infiriendo los tipos estáticos directamente del esquema en tiempo de ejecución.[2]

### Integración mediante Resolvers

La librería @hookform/resolvers actúa como el puente entre RHF y Zod. El resolver transforma la salida de la validación de Zod en el formato de errores que RHF espera, manejando la asignación de mensajes de error a los campos correspondientes.[2]

**Beneficios de esta integración:**

- **Fuente Única de Verdad:** El esquema Zod define tanto las reglas de validación (runtime) como los tipos de TypeScript (compile-time).
- **Desacoplamiento:** La lógica de validación se separa de la UI. El mismo esquema puede reutilizarse en el backend para validar la petición API o Server Action, garantizando

consistencia isomórfica.[2]

## 2.3 Patrones Avanzados de Componentes Reutilizables (Generic Forms)

Uno de los mayores retos reportados por los equipos de desarrollo es la creación de componentes de formulario genéricos que mantengan la seguridad de tipos. Al abstraer un <FormInput>, a menudo se pierde la conexión entre el prop name y el tipo del formulario padre.

### El Problema de los Tipos Anidados

Cuando trabajamos con formularios anidados (ej. user.address.street), pasar cadenas de texto simples como name="address.street" puede ser validado por RHF, pero TypeScript a menudo pierde la capacidad de autocompletar o validar que esa ruta existe en el esquema, especialmente dentro de componentes hijos genéricos.[5]

### Solución: Tipos de Marca (Branded Types) y Generics

Para resolver esto, utilizamos una combinación de genéricos en TypeScript y utilidades de RHF como Path<T>, Control<T>, y UseFormRegister<T>. Además, existe un patrón avanzado utilizando **tipos de marca (branded types)** para manipular rutas anidadas con seguridad total.[6]

Un patrón robusto para un componente *wrapper* genérico sería:

TypeScript

```
import { useForm, UseFormReturn, SubmitHandler, FieldValues, Path, Control } from "react-hook-form";
```

```
import { zodResolver } from "@hookform/resolvers/zod";
import { z, ZodType } from "zod";

// Definición de Props para el Wrapper del Formulario
interface SmartFormProps<T extends FieldValues> {
  schema: ZodType<T>;
  onSubmit: SubmitHandler<T>;
  children: (methods: UseFormReturn<T>) => React.ReactNode;
  defaultValues?: DefaultValues<T>;
}

export const SmartForm = <T extends FieldValues>({
  schema,
  onSubmit,
  children,
  defaultValues
}: SmartFormProps<T>) => {
  const methods = useForm<T>({
    resolver: zodResolver(schema),
    defaultValues, // Importante: Evitar undefined para inputs controlados [7]
  });

  return (
    <form onSubmit={methods.handleSubmit(onSubmit)}>
      {children(methods)}
    </form>
  );
};
```

Para los inputs individuales, el uso de Control es preferible a register cuando se envuelven componentes de UI complejos (como los de librerías de componentes tipo Shadcn/ui o Material UI), ya que Controller maneja la lógica de puente entre el estado no controlado de RHF y el estado controlado que suelen requerir estos componentes.[8]

TypeScript

```
// Input Tipado Genérico
interface TypedInputProps<T extends FieldValues> {
  name: Path<T>; // Clave para asegurar que el string es una ruta válida en T
```

```
  control: Control<T>;
  label: string;
}

const TypedInput = <T extends FieldValues>({ name, control, label }: TypedInputProps<T>) => {
  return (
    <Controller
      name={name}
      control={control}
      render={({ field, fieldState: { error } }) => (
        <div>
          <label>{label}</label>
          <input {...field} aria-invalid={!!error} />
           {/* Accesibilidad: vincular mensaje de error con input */}
          {error && <span role="alert">{error.message}</span>}
        </div>
      )}
    />
  );
};
```

## 2.4 Accesibilidad (A11y) en Formularios Modernos

La accesibilidad no es una capa opcional, sino un requisito fundamental. RHF soporta validación nativa del navegador, pero al usar validación personalizada con Zod, debemos gestionar manualmente los atributos ARIA para asegurar que los lectores de pantalla comuniquen los errores correctamente.[9]

**Prácticas Esenciales:**

1. **aria-invalid:** Debe establecerse en true cuando el campo tiene un error en formState.errors.
2. **aria-describedby:** Debe apuntar al ID del elemento que contiene el mensaje de error textual.
3. **Foco en Error:** RHF maneja automáticamente el foco en el primer campo con error tras un intento de envío fallido (shouldFocusError: true por defecto), lo cual es vital para la navegación por teclado.

## 2.5 Integración con Next.js Server Actions

La introducción de las **Server Actions** en Next.js ha redefinido el flujo de envío de datos. Ya no es estrictamente necesario crear endpoints API REST o GraphQL intermedios; el formulario puede invocar directamente una función asíncrona en el servidor.

### El Desafío de la Validación Isomórfica

El objetivo es validar en el cliente para feedback inmediato y en el servidor para seguridad. Usar el mismo esquema Zod en ambos lados es la clave.[10]

**Patrón de Implementación con useActionState (anteriormente useFormState):**

1. **Cliente:** RHF valida usando zodResolver. Si falla, detiene el envío.
2. **Envío:** Si pasa, se dispara la Server Action.
3. **Servidor:** La Server Action recibe FormData. Debe parsearlo y validarlo nuevamente con Zod (schema.safeParse).
   - Si la validación falla (ej. dato manipulado o regla de negocio compleja como "email ya registrado"), la acción retorna un objeto de estado con errores serializados.
4. **Hidratación de Errores:** En el cliente, un useEffect observa el estado devuelto por la acción y utiliza setError de RHF para inyectar los errores del servidor en los campos correspondientes, haciendo que parezcan errores nativos del formulario.[11]

TypeScript

```typescript
// Server Action (actions.ts)
'use server'
export async function createItem(prevState: any, formData: FormData) {
  const data = Object.fromEntries(formData);
  const result = schema.safeParse(data);

  if (!result.success) {
    return {
      errors: result.error.flatten().fieldErrors, // Serializar errores de Zod
      message: "Error de validación"
    };
```

```
  }
  //... lógica de base de datos...
}
```

TypeScript

```
// Cliente
const [state, formAction] = useActionState(createItem, initialState);
const { setError } = useForm();

useEffect(() => {
  if (state?.errors) {
    Object.entries(state.errors).forEach(([key, msgs]) => {
      setError(key as any, { type: 'server', message: msgs });
    });
  }
}, [state, setError]);
```

Este patrón cierra el ciclo, ofreciendo una UX robusta donde los errores de backend se sienten instantáneos e integrados.[12]

---

# 3. Visualización y Manipulación de Datos: TanStack Table v8

**TanStack Table** (anteriormente React Table) representa un cambio de paradigma: es una librería "headless" (sin cabeza). Esto significa que no renderiza markup (ni <table>, ni divs, ni estilos). Su única responsabilidad es gestionar la lógica del estado de la tabla (ordenamiento, filtrado, paginación, selección) y exponer una API para que el desarrollador construya la UI.[13] Esto otorga un control total sobre el diseño, permitiendo integrar la tabla con cualquier sistema de diseño (Tailwind, Material UI, Bootstrap).

## 3.1 Definición de Columnas y Seguridad de Tipos (TData)

En la versión 8, la seguridad de tipos es una prioridad. Definir columnas utilizando objetos literales simples puede ser propenso a errores y perder la inferencia de tipos. La práctica recomendada es utilizar el utilitario **createColumnHelper**.[14]

### Column Helper vs. Accessor Keys

El uso de createColumnHelper permite crear definiciones de columnas que están fuertemente tipadas contra la interfaz de datos (TData).

TypeScript

```
type Usuario = {
  id: string;
  nombre: string;
  detalles: {
    edad: number;
    email: string;
  }
};

const columnHelper = createColumnHelper<Usuario>();

const columns =;
```

Este enfoque previene errores comunes como errores tipográficos en las claves de acceso (accessorKey), ya que TypeScript validará que la cadena proporcionada sea una ruta válida dentro del objeto Usuario.[16]

## 3.2 Estrategias de Filtrado: Global y por Columna

El filtrado es una funcionalidad crítica que puede impactar severamente el rendimiento si no

se gestiona adecuadamente. TanStack Table soporta filtrado tanto en cliente como en servidor.

### Filtrado Global en Cliente

Para datasets pequeños o medianos cargados en memoria, TanStack Table ofrece algoritmos de filtrado integrados (como includesString). Es necesario configurar getFilteredRowModel en la instancia de la tabla.[17]

Un aspecto crucial para la UX es la implementación de un **input con Debounce**. Filtrar en cada pulsación de tecla (onChange directo) causa re-renderizados masivos y recálculos costosos de la tabla.

Implementación de DebouncedInput:
El componente DebouncedInput mantiene un estado local para el valor del input y solo propaga el cambio al estado de la tabla (o URL) después de un periodo de inactividad (ej. 300-500ms).19

TypeScript

```
// DebouncedInput.tsx (Resumen del patrón)
function DebouncedInput({ value: initialValue, onChange, debounce = 500,...props }) {
  const [value, setValue] = useState(initialValue);

  useEffect(() => {
    setValue(initialValue);
  }, [initialValue]);

  useEffect(() => {
    const timeout = setTimeout(() => {
      onChange(value);
    }, debounce);
    return () => clearTimeout(timeout);
  }, [value, debounce]); // Solo dispara onChange tras el delay

  return <input {...props} value={value} onChange={e => setValue(e.target.value)} />;
}
```

Este componente es esencial para evitar lag en la interfaz y, en el caso de filtrado en servidor, para prevenir el "bombardeo" de peticiones a la API.[21]

## 3.3 Paginación: Cliente vs. Servidor

La decisión entre paginación en cliente o servidor define la arquitectura de datos de la vista.

| Característica | Paginación Cliente | Paginación Servidor |
|---|---|---|
| **Volumen de Datos** | Pequeño (< 10k filas aprox) | Grande / Infinito |
| **Carga Inicial** | Lenta (trae todo el JSON) | Rápida (trae solo página 1) |
| **Interacción** | Instantánea (ordenar/filtrar en memoria) | Latencia de red por acción |
| **Configuración Tabla** | getPaginationRowModel | manualPagination: true |

### Implementación de Paginación en Servidor

Cuando se opta por paginación manual (servidor), la tabla no conoce el dataset completo, solo la "página" actual. Por tanto, es vital pasar la propiedad pageCount a la instancia de la tabla, calculada en base al total de registros que reporta el backend, para que la tabla pueda renderizar los controles de paginación correctamente (saber si existe una página siguiente).[13]

El estado de paginación (pageIndex, pageSize) debe ser controlado y, idealmente, sincronizado con la URL, como veremos en la siguiente sección.

TypeScript

```
const table = useReactTable({
```

```
  data, // Solo los datos de la página actual
  columns,
  pageCount: data?.totalPages?? -1, // -1 permite paginación infinita si se desconoce el total
  state: {
    pagination: { pageIndex, pageSize },
  },
  manualPagination: true, // Desactiva la lógica interna de paginación
  onPaginationChange: setPagination, // Hook para actualizar estado/URL
  //...
});
```

.[22]

---

# 4. Gestión de Estado en la URL: La Fuente de la Verdad

El principio rector de esta arquitectura moderna es: **Si el estado afecta lo que el usuario ve y debe ser compartible, pertenece a la URL.** Esto incluye filtros de búsqueda, página actual, ordenamiento de columnas y pestañas activas.

## 4.1 Limitaciones de useSearchParams Nativo

Tanto React Router como Next.js proporcionan hooks (useSearchParams) para leer la query string. Sin embargo, trabajar directamente con ellos presenta fricciones significativas [24]:

1. **Todo es String:** ?page=5 devuelve "5". Requiere casteo manual (Number()) y manejo de NaN.
2. **Serialización Manual:** Guardar objetos complejos o arrays requiere lógica repetitiva de JSON.stringify o manipulación de múltiples claves.
3. **Actualizaciones Imperativas:** Actualizar un parámetro a menudo implica crear una nueva instancia de URLSearchParams, modificarla y empujarla al router, lo que resulta en código verboso.
4. **Hidratación:** En Next.js, sincronizar el estado del servidor (Server Components) con el estado del cliente inicial puede causar errores de hidratación si no se maneja con cuidado.

## 4.2 La Solución Type-Safe: nuqs

La librería **nuqs** (Next.js URL Query State) emerge como la solución definitiva para este problema. Proporciona una API similar a useState pero que persiste, lee y escribe directamente en la URL, con seguridad de tipos completa.[26]

### Parsers y Seguridad de Tipos

nuqs utiliza el concepto de "parsers" para definir el tipo de dato esperado en la URL. Esto elimina la necesidad de casteo manual y proporciona valores por defecto seguros.

TypeScript

```
import { useQueryState, parseAsInteger, parseAsString } from 'nuqs';

export function ComponenteTabla() {
  // Estado sincronizado: Lee de URL, escribe en URL
  const [page, setPage] = useQueryState(
    'page',
    parseAsInteger.withDefault(1) // Si?page= no existe o es inválido, devuelve 1
  );

  const = useQueryState(
    'q',
    parseAsString
     .withDefault('')
     .withOptions({ throttleMs: 500 }) // Throttling integrado para inputs rápidos
  );

  // setPage(2) actualiza la URL a?page=2 automáticamente
}
```

### Gestión del Historial y Throttling

Una ventaja crítica de nuqs sobre implementaciones manuales es el control sobre la pila del historial del navegador. Podemos decidir si una actualización de estado debe crear una nueva entrada en el historial (history: 'push') o reemplazar la actual (history: 'replace').

- **Replace:** Ideal para inputs de búsqueda o scroll infinito (no queremos que el usuario tenga que dar "Atrás" 50 veces por cada letra escrita).
- **Push:** Ideal para cambios significativos como cambiar de página o aplicar un filtro de categoría importante.
- **Throttling:** Evita que el navegador bloquee actualizaciones rápidas de URL (Safari limita las actualizaciones de historial a ~100 por 30 segundos). nuqs maneja esto internamente.[28]

### 4.3 SEO y Compartibilidad

Al elevar el estado a la URL, mejoramos automáticamente el SEO (para páginas públicas) y la experiencia de compartir. Un enlace enviado por email llevará al destinatario exactamente a la vista de datos configurada por el remitente, sin necesidad de instrucciones adicionales ("ve a la página 3 y filtra por 'Activos'").[29]

---

## 5. Arquitectura Integrada: El Flujo de Datos Completo

La verdadera potencia de estas tecnologías se revela cuando se orquestan juntas. A continuación, describimos el flujo de datos recomendado para una vista de "Dashboard de Administración" en una aplicación Next.js moderna.

### 5.1 El Ciclo de Vida de una Petición

1. **Entrada (Server Component):**
    - La página recibe searchParams como prop.
    - Estos parámetros (que son strings crudos) se parsean usando una definición de esquema de caché de nuqs (para type-safety en el servidor).
    - Se realiza la consulta a la base de datos (ORM) utilizando estos parámetros validados

para filtrar y paginar los datos de manera eficiente.
    - Se renderiza el componente Cliente <DataTable />, pasando los datos iniciales y el estado de paginación calculado.
2. **Inicialización (Client Component - TanStack Table + nuqs):**
    - La tabla se inicializa. En lugar de usar useState para pagination o sorting, utilizamos hooks de nuqs (useQueryState).
    - Esto crea una **Single Source of Truth**. La tabla lee su configuración directamente de la URL. Si la URL cambia (por navegación atrás/adelante), la tabla se actualiza reactivamente.
3. **Interacción y Filtrado:**
    - El usuario escribe en el <DebouncedInput /> vinculado al estado globalFilter (gestionado por nuqs).
    - Tras el debounce, nuqs actualiza la URL (?q=react).
    - **Reactividad:** En Next.js App Router, esta actualización de URL puede configurarse para disparar un re-renderizado del Server Component (shallow: false), lo que provoca que el servidor vuelva a ejecutar la consulta a la base de datos con el nuevo filtro y envíe los nuevos datos a la tabla. Alternativamente, para una experiencia más "app-like", se puede usar React Query hidratado con los nuevos parámetros para hacer el fetch en segundo plano.
4. **Mutación (React Hook Form):**
    - El usuario selecciona una fila para editar. Se abre un formulario controlado por RHF + Zod.
    - Al guardar, se ejecuta una Server Action.
    - Si es exitosa, la Server Action llama a revalidatePath('/dashboard').
    - Next.js invalida la caché de la ruta, recarga los datos de la tabla (manteniendo la paginación y filtros de la URL intactos) y muestra la información actualizada.

## 5.2 Conclusión y Recomendaciones Finales

La convergencia de **Zod** para la integridad de datos, **React Hook Form** para la captura eficiente, **TanStack Table** para la visualización flexible, y **nuqs** para la persistencia de estado en la URL, constituye la arquitectura de referencia para aplicaciones React modernas de alta complejidad.

Adoptar este stack implica una curva de aprendizaje, especialmente en el dominio de los tipos genéricos de TypeScript y la mentalidad "URL-first". Sin embargo, el retorno de inversión es inmenso:

- **Código más limpio:** Eliminación de efectos secundarios manuales (useEffect) para sincronizar estados.

- **Menos bugs:** La seguridad de tipos de extremo a extremo (desde la URL hasta la base de datos y de vuelta al formulario) previene errores de tiempo de ejecución.
- **Mejor UX:** Aplicaciones rápidas, compartibles y que respetan las expectativas de navegación del usuario.

Para los líderes técnicos, la recomendación es estandarizar estos patrones mediante la creación de componentes wrapper internos (como SmartForm o DataTable configurado) que encapsulen la complejidad de la configuración, permitiendo a los desarrolladores de producto centrarse en la lógica de negocio.

---

## Referencias Integradas

- **Gestión de Formularios:**.[30]
- **Tablas y Datos:**.[13]
- **Estado en URL:**.[24]

### Obras citadas


1. React Hook Form with Zod Validation: A Complete Guide with TypeScript | by Toukir Ahamed Pigeon | Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/@toukir.ahamed.pigeon/react-hook-form-with-zod-validation-a-complete-guide-with-typescript-aacbcb370a8b
2. Learn Zod validation with React Hook Form - Contentful, fecha de acceso: noviembre 29, 2025, https://www.contentful.com/blog/react-hook-form-validation-zod/
3. Form Schema Validation with Zod and React Hook Form - Refine dev, fecha de acceso: noviembre 29, 2025, https://refine.dev/blog/zod-typescript/
4. react-hook-form/resolvers: Validation resolvers: Yup, Zod ... - GitHub, fecha de acceso: noviembre 29, 2025, https://github.com/react-hook-form/resolvers
5. Improve TypeScript Generics Handling to Allow For Reusable Components · Issue #11617 · react-hook-form/react-hook-form - GitHub, fecha de acceso: noviembre 29, 2025, https://github.com/react-hook-form/react-hook-form/issues/11617
6. V8: Reusable components in TypeScript (using branded strings ..., fecha de acceso: noviembre 29, 2025, https://github.com/react-hook-form/react-hook-form/discussions/7354
7. Simplify Your Form Handling with a Custom Wrapper for react-use-form - Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/@vladkens/simplify-your-form-handling-with-a-custom-wrapper-for-react-use-form-55add66c9a21
8. Advanced Usage - React Hook Form, fecha de acceso: noviembre 29, 2025, https://react-hook-form.com/advanced-usage

9. Server Actions and Mutations - Data Fetching - Next.js, fecha de acceso: noviembre 29, 2025, https://nextjs.org/docs/14/app/building-your-application/data-fetching/server-actions-and-mutations
10. React Hook Form: Client & Server Validation with Zod in Next.js - YouTube, fecha de acceso: noviembre 29, 2025, https://www.youtube.com/watch?v=c3IgVJ3jnmQ
11. Handling Forms in Next.js with React Hook Form, Zod, and Server Actions. - Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/@techwithtwin/handling-forms-in-next-js-with-react-hook-form-zod-and-server-actions-e148d4dc6dc1
12. Pagination Guide | TanStack Table Docs, fecha de acceso: noviembre 29, 2025, https://tanstack.com/table/v8/docs/guide/pagination
13. Migrating to V8 Guide | TanStack Table Docs, fecha de acceso: noviembre 29, 2025, https://tanstack.com/table/v8/docs/guide/migrating
14. Columns Guide | TanStack Table Docs, fecha de acceso: noviembre 29, 2025, https://tanstack.com/table/v8/docs/guide/column-defs
15. TanStack Table v8: TypeScript; Do NOT use AccessorKey, instead ALWAYS use columnHelper.accessor - YouTube, fecha de acceso: noviembre 29, 2025, https://www.youtube.com/watch?v=eYxVVxEcDdI
16. Global Filtering APIs | TanStack Table Docs, fecha de acceso: noviembre 29, 2025, https://tanstack.com/table/v8/docs/api/features/global-filtering
17. Global Filtering Guide | TanStack Table Docs, fecha de acceso: noviembre 29, 2025, https://tanstack.com/table/v8/docs/guide/global-filtering
18. React TanStack Table Filters Example | TanStack Table Docs, fecha de acceso: noviembre 29, 2025, https://tanstack.com/table/v8/docs/framework/react/examples/filters
19. React TanStack Table Filters Example, fecha de acceso: noviembre 29, 2025, https://tanstack.com/table/latest/docs/framework/react/examples/filters
20. Create a reusable react-table component with Typescript - DEV Community, fecha de acceso: noviembre 29, 2025, https://dev.to/esponges/create-a-reusable-react-table-component-with-typescript-56d4
21. Pagination APIs | TanStack Table Docs, fecha de acceso: noviembre 29, 2025, https://tanstack.com/table/v8/docs/api/features/pagination
22. Server side pagination with V8: `getCanNextPage` issue. · TanStack table · Discussion #3822 - GitHub, fecha de acceso: noviembre 29, 2025, https://github.com/TanStack/table/discussions/3822
23. Why URL state matters: A guide to useSearchParams in React - LogRocket Blog, fecha de acceso: noviembre 29, 2025, https://blog.logrocket.com/url-state-usesearchparams/
24. Understanding useParams() and useSearchParams() in React Router — A Beginner's Guide | by Soham Dave | Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/@SohamDave08/understanding-useparams-and-usesearchparams-in-react-router-a-beginners-guide-6faaf0534187

25. Why You Should Use nuqs: Smarter URL State Management for React & Next.js - Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/@ruverd/why-you-should-use-nuqs-smarter-url-state-management-for-react-next-js-26a8b51ca1ac
26. nuqs | Type-safe search params state management for React, fecha de acceso: noviembre 29, 2025, https://nuqs.dev/
27. The BEST Type-Safe URL State Manager - YouTube, fecha de acceso: noviembre 29, 2025, https://www.youtube.com/watch?v=t4d1cJarJqs
28. Nuqs: Because URLs Should Do More | by Lope - Medium, fecha de acceso: noviembre 29, 2025, https://medium.com/@iamlope/nuqs-because-urls-should-do-more-5d5d86e873c1
29. How to create forms with Server Actions - Next.js, fecha de acceso: noviembre 29, 2025, https://nextjs.org/docs/app/guides/forms
30. 47ng/nuqs: Type-safe search params state manager for React frameworks - Like useState, but stored in the URL query string. - GitHub, fecha de acceso: noviembre 29, 2025, https://github.com/47ng/nuqs